TEXCOMS WORLDWIDE
IT Department — Confidential

DESIGN DOCUMENT

Project Expenses Tracker

Thailand Fieldtrip Edition

thai.expenses.syaefulaz.my.id

Document ID	DD-TXC-EXP-2025-001
Version	v1.0 — Initial Design
Date	May 2025
Domain	thai.expenses.syaefulaz.my.id
Classification	Internal — Confidential
Author	IT Head, Texcoms Worldwide
Status	For Design Review & Approval

1. Document Purpose

This Design Document (DD) provides the technical architecture, system design, and implementation specification for the Project Expenses Tracker system. It complements the BRD (BRD-TXC-EXP-2025-001) by detailing how the requirements — including the Loan & Settlement feature — will be implemented. This document is intended for the development team and for review by the Finance Manager and Director as part of the design sign-off process.

2. System Overview

2.1 Architecture Overview

The system follows a 3-tier architecture: Presentation Layer, Application Layer, and Data Layer. Nginx serves as a reverse proxy and SSL terminator in front of the NodeJS API. Angular compiles to static assets served directly by Nginx.

Layer	Technology	Notes
Presentation	Angular 21 + PrimeNG	TypeScript, RxJS, Angular Router
Application	NodeJS 18 + Express/NestJS	REST API, TypeScript, JWT auth
Data	MySQL 8	InnoDB engine, transactions, FK constraints
Web Server	Nginx 1.24+	Reverse proxy, SSL termination, static file serving
Authentication	JWT	Access token (15-min) + Refresh token (7-day)

2.2 High-Level Data Flow

Step	Actor / Component	Action
1	User (Dina / Syaeful / Winda)	Opens browser → https://thai.expenses.syaefulaz.my.id
2	Nginx	Terminates HTTPS; proxies /api/* to NodeJS; serves Angular static assets
3	Angular SPA	Renders dashboard; user creates expense with source field
4	NodeJS API	Validates input; if source belongs to another user → creates loan record
5	MySQL	Stores expense + loan in same transaction; returns confirmation
6	Lender Dashboard	Shows outstanding loan with declare/repay controls

2.3 Technology Stack

Component	Technology	Version	Notes
UI Framework	Angular	21.x	TypeScript, RxJS, Angular Router
Component Library	PrimeNG	18.x	Table, Form, Dialog, Chart components
Backend Runtime	NodeJS	18.x LTS	JavaScript runtime
API Framework	Express / NestJS	5.x / 10.x	REST API, TypeScript
ORM	TypeORM / Prisma	Latest	Database abstraction layer

Component	Technology	Version	Notes
Database	MySQL	8.x	InnoDB engine, transactions
Web Server	Nginx	1.24+	Reverse proxy, SSL
SSL	Let's Encrypt	Certbot	Auto-renewal via cron
Authentication	JWT	—	Access token + refresh token
Password Storage	bcrypt	—	Cost factor 12

3. Frontend Design

3.1 Page Structure

Page	Route	Description
Login	/login	Email + password authentication
Dashboard	/dashboard	KPIs, budget charts, loan summary widget
Expense List	/expenses	Filterable/sortable expense table
Expense Form	/expenses/new	Create expense: date, amount, category, source, etc.
Expense Detail	/expenses/:id	View / edit expense record
Project List	/projects	Manage projects and budgets
Loan Dashboard	/loans	Outstanding loans: owed to me / I owe others [NEW]
Approval Queue	/approvals	Pending approvals (Finance Staff / Manager)
User Management	/admin/users	User CRUD (Admin only)
Settings	/admin/settings	Categories, cost centers config
Reports	/reports	PDF / Excel export interface
Executive View	/executive	Director-only summary dashboard

3.2 Navigation Structure

Element	Content
Top Nav Bar	Logo | Dashboard | Expenses | Projects | Loans | Approvals | Reports | Executive | Admin ▼
Loan Widget (persistent)	Always visible in nav — shows total owed to me / I owe others (badge)
Footer	© 2025 Texcoms Worldwide | Logged in as [User] | Logout

3.3 Key PrimeNG Components

Component	PrimeNG Module	Usage
Data Table	TableModule	Expense list with sort, filter, pagination, row expansion
Form Fields	InputText, InputNumber, Dropdown, Calendar	Expense entry and edit forms
Source Selector	Dropdown (custom)	Select source: Winda Cash, ATM Syaeful, Dina Personal, etc.
Loan Widget	Card + Badge + Table	Persistent dashboard widget for outstanding loans
Dialog	DialogModule	Confirmation dialogs, expense detail modal
Charts	ChartModule (Chart.js)	Budget utilization donut, MTD bar chart
Toast / Messages	MessageModule	Success / error feedback notifications
Breadcrumb	BreadcrumbModule	Navigation context trail
File Upload	FileUploadModule	Receipt / image attachment upload
Progress Bar	ProgressBarModule	Budget utilization visual indicator

3.4 Expense Entry Form Fields

Field	Type	Required	Notes
expense_date	Calendar (date picker)	Yes	Date when expense occurred — NOT submission date
amount	InputNumber	Yes	IDR; two decimal precision
category	Dropdown	Yes	e.g. Transport, Food, Accommodation, Other
description	InputTextarea	No	Free-text description of the expense
source	Dropdown	Yes	Who funded: Winda Cash, ATM Syaeful, Dina Personal, Corporate ATM
project	Dropdown	Yes	Linked project
cost_center	Dropdown	Yes	Linked cost center
attachment	FileUpload	No	Receipt or proof of expense (image / PDF)

3.5 Loan Dashboard [NEW]

The Loan Dashboard (route: /loans) shows the logged-in user's position as both lender and borrower. It is the primary UI for the Loan & Settlement feature (FR-09 through FR-13).

Section	Description
Money I Lent	List of unsettled loans where the logged-in user is the lender. Shows: borrower name, original amount (THB), declared repayment, actual repaid, remaining balance, status badge. Actions: Declare Repayment Amount, Record Actual Repayment.
Money I Borrowed	List of loans where the logged-in user is the borrower. Shows: lender name, amount, remaining balance. Action: None — only the lender records repayments.
Summary Cards	Total outstanding lent, total outstanding borrowed, net position

3.6 Responsive Breakpoints

Breakpoint	Width	Layout
Desktop	>= 1200px	Full sidebar + content layout
Tablet	768px – 1199px	Collapsible sidebar
Mobile	< 768px	Stacked layout, hamburger menu

4. Backend Design (NodeJS 18)

4.1 API Architecture

The backend exposes a RESTful API built with Express or NestJS (TypeScript). All endpoints are prefixed with /api/v1/. Authentication uses JWT access tokens (15-minute expiry) and refresh tokens (7-day expiry). All passwords are hashed with bcrypt (cost factor 12).

4.2 API Endpoints

Method	Endpoint	Description	Auth
POST	/api/v1/auth/login	Authenticate user, return JWT	Public
POST	/api/v1/auth/refresh	Refresh access token	Refresh Token
POST	/api/v1/auth/logout	Invalidate refresh token	JWT
GET	/api/v1/expenses	List expenses (filtered, paginated)	JWT
POST	/api/v1/expenses	Create expense — if source != user, auto-create loan	JWT
GET	/api/v1/expenses/:id	Get expense detail	JWT
PUT	/api/v1/expenses/:id	Update expense (Draft only)	JWT
DELETE	/api/v1/expenses/:id	Delete expense (Draft only)	JWT
POST	/api/v1/expenses/:id/submit	Submit expense for approval	JWT
POST	/api/v1/expenses/:id/approve	Finance approve / reject	JWT (FIN/FM)
GET	/api/v1/loans	List all loans for current user (as lender + borrower)	JWT
GET	/api/v1/loans/:id	Get loan detail	JWT
PUT	/api/v1/loans/:id/declare	Lender declares expected repayment amount	JWT (Lender only)
PUT	/api/v1/loans/:id/repay	Lender records actual repayment (partial or full)	JWT (Lender only)
GET	/api/v1/loans/summary	Summary: total lent, total borrowed, net position	JWT
GET	/api/v1/projects	List projects	JWT
POST	/api/v1/projects	Create project	JWT (ADMIN)
GET	/api/v1/projects/:id/budget	Budget utilization for project	JWT
GET	/api/v1/categories	List categories	JWT

Method	Endpoint	Description	Auth
POST	/api/v1/categories	Create category	JWT (ADMIN)
GET	/api/v1/cost-centers	List cost centers	JWT
GET	/api/v1/dashboard	Dashboard KPIs including loan summary	JWT
GET	/api/v1/users	List users	JWT (ADMIN)
POST	/api/v1/users	Create user	JWT (ADMIN)
GET	/api/v1/reports/expenses	Expense report data	JWT
GET	/api/v1/reports/export/pdf	Export PDF report	JWT
GET	/api/v1/reports/export/excel	Export Excel report	JWT

4.3 Standard API Response Format

Success:

{"success": true, "data": { ... }, "meta": {"page": 1, "limit": 20, "total": 100}}

Error:

{"success": false, "error": {"code": "EXPENSE_NOT_FOUND", "message": "Expense with ID 5 not found."}}

4.4 Key Request / Response Schemas

Create Expense (POST /api/v1/expenses)

Field	Type	Required	Notes
project_id	INT	Yes	FK → projects.id
cost_center_id	INT	Yes	FK → cost_centers.id
category_id	INT	Yes	FK → categories.id
expense_date	DATE (YYYY-MM-DD)	Yes	Date expense occurred — NOT submission date
amount	DECIMAL(15,2)	Yes	Amount in IDR
description	TEXT	No	Free-text description
source	VARCHAR(100)	Yes	Source of funds: Winda Cash, ATM Syaeful, Dina Personal, etc.
attachment	FILE	No	Receipt image / PDF

Loan Record (auto-created on expense submission)

Field	Type	Description
id	INT	Primary key

Field	Type	Description
expense_id	INT	FK → expenses.id — links loan to source expense
lender_id	INT	FK → users.id — person whose cash/funds were used
borrower_id	INT	FK → users.id — person who made the expense
amount	DECIMAL(15,2)	Original loan amount (THB)
declared_repayment	DECIMAL(15,2)	Amount borrower says they will repay (nullable)
actual_repaid	DECIMAL(15,2)	Amount actually repaid so far (default 0)
remaining_balance	DECIMAL(15,2)	declared_repayment – actual_repaid (computed)
status	ENUM('UNSETTLED','PARTIAL','FULLY_SETTLED','OVERPAID')	Loan lifecycle state
created_at	TIMESTAMP	When loan was created
updated_at	TIMESTAMP	Last repayment update

5. Database Design (MySQL 8)

The Entity-Relationship summary below shows the complete schema including the new loans and loan_repayments tables. Primary keys and foreign keys are enforced at the database level.

5.1 Entity-Relationship Summary

Relationship	From	To	Type	Notes
Creates	User	Expense	1 : N	A user can create many expenses
Funds	User	Loan (as lender)	1 : N	A user can be lender on many loans
Borrows	User	Loan (as borrower)	1 : N	A user can owe many loans
Originates	Expense	Loan	1 : 1	Each loan stems from exactly one expense
Belongs to	Expense	Project	N : 1	Each expense belongs to one project
Tracks	Expense	ApprovalLog	1 : N	Each expense has multiple approval steps

5.2 Table Schemas

users

Column	Type	Constraints	Notes
id	INT	PK, AUTO_INCREMENT	Primary key
name	VARCHAR(100)	NOT NULL	Full display name (e.g. Dina, Syaeful, Winda)
email	VARCHAR(255)	UNIQUE, NOT NULL	Login email address
password_hash	VARCHAR(255)	NOT NULL	bcrypt hashed password
role	ENUM('PM','FIN','FM','ADMIN','DIR')	NOT NULL	User role code
is_active	BOOLEAN	DEFAULT TRUE	Soft deactivation flag
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	
updated_at	TIMESTAMP	ON UPDATE CURRENT_TIMESTAMP	

NEW loans [NEW TABLE]

Column	Type	Constraints	Notes
id	INT	PK, AUTO_INCREMENT	Primary key
expense_id	INT	FK → expenses.id, UNIQUE	One loan per expense (1:1)
lender_id	INT	FK → users.id, NOT NULL	User whose source funds were used
borrower_id	INT	FK → users.id, NOT NULL	User who recorded the expense
amount	DECIMAL(15,2)	NOT NULL	Original loan amount (THB)
declared_repayment	DECIMAL(15,2)	NULL	Amount borrower commits to repay
actual_repaid	DECIMAL(15,2)	DEFAULT 0, NOT NULL	Amount actually repaid so far
remaining_balance	DECIMAL(15,2)	Computed: declared – actual	Displayed on dashboards
status	ENUM('UNSETTLED','PARTIAL','FULLY_SETTLED','OVERPAID')	NOT NULL, DEFAULT UNSETTLED	Lifecycle state
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	

Column	Type	Constraints	Notes
updated_at	TIMESTAMP	ON UPDATE CURRENT_TIMESTAMP	

NEW loan_repayments [NEW TABLE]

Column	Type	Constraints	Notes
id	INT	PK, AUTO_INCREMENT	Primary key
loan_id	INT	FK → loans.id, NOT NULL	Associated loan
recorded_by_id	INT	FK → users.id, NOT NULL	Lender who recorded the repayment
amount	DECIMAL(15,2)	NOT NULL	Repayment amount (THB)
note	TEXT	NULL	Optional note (e.g. "Partial payment — 500 THB")
repaid_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	

expenses

Column	Type	Constraints	Notes
id	INT	PK, AUTO_INCREMENT	Primary key
user_id	INT	FK → users.id, NOT NULL	User who created the expense
project_id	INT	FK → projects.id, NOT NULL	Linked project
cost_center_id	INT	FK → cost_centers.id, NOT NULL	Linked cost center
category_id	INT	FK → categories.id, NULL	Expense category
expense_date	DATE	NOT NULL	Date expense occurred — distinct from created_at
amount	DECIMAL(15,2)	NOT NULL	Amount in IDR
description	TEXT	NULL	Free-text description
source	VARCHAR(100)	NOT NULL	Source of funds used (e.g. Winda Cash)

Column	Type	Constraints	Notes
attachment_path	VARCHAR(500)	NULL	Path to receipt file
status	ENUM('DRAFT','PENDING','APPROVED','FINAL_APPROVED','REJECTED','CLOSED')	NOT NULL, DEFAULT DRAFT	Workflow status
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	
updated_at	TIMESTAMP	ON UPDATE CURRENT_TIMESTAMP	

projects

Column	Type	Constraints	Notes
id	INT	PK, AUTO_INCREMENT	Primary key
code	VARCHAR(20)	UNIQUE, NOT NULL	Project code
name	VARCHAR(200)	NOT NULL	Project name
start_date	DATE	NOT NULL	
end_date	DATE	NOT NULL	
total_budget	DECIMAL(15,2)	NOT NULL, DEFAULT 0	Budget in IDR
is_active	BOOLEAN	DEFAULT TRUE	
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	
updated_at	TIMESTAMP	ON UPDATE CURRENT_TIMESTAMP	

cost_centers

Column	Type	Constraints	Notes
id	INT	PK, AUTO_INCREMENT	Primary key
name	VARCHAR(100)	NOT NULL	Cost center name
code	VARCHAR(20)	UNIQUE, NOT NULL	Cost center code
is_active	BOOLEAN	DEFAULT TRUE	
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	

categories

Column	Type	Constraints	Notes
id	INT	PK, AUTO_INCREMENT	Primary key
name	VARCHAR(100)	NOT NULL	Category name (Transport, Food, Accommodation, Other)
parent_id	INT	FK → categories.id, NULL	Self-referential for subcategories
is_active	BOOLEAN	DEFAULT TRUE	
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	

approval_logs

Column	Type	Constraints	Notes
id	INT	PK, AUTO_INCREMENT	Primary key
expense_id	INT	FK → expenses.id, NOT NULL	Associated expense
actor_id	INT	FK → users.id, NOT NULL	User who performed the action
action	ENUM('SUBMIT','APPROVE','REJECT')	NOT NULL	Action taken
comment	TEXT	NULL	Approval / rejection comment
acted_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	

6. Security Design

Area	Implementation
HTTPS	SSL/TLS enforced on all traffic; HTTP redirects to HTTPS
Password Storage	bcrypt cost factor 12; no plaintext passwords ever stored
Authentication	JWT access tokens (15-min expiry) + refresh tokens (7-day, httpOnly cookie)
Authorization	RBAC enforced at API layer; role checked on every protected endpoint
Input Validation	Joi / class-validator schema validation on all API inputs
SQL Injection	Parameterized queries via TypeORM / Prisma; no raw SQL string concatenation
XSS	Angular's built-in DOM sanitization; Content-Security-Policy header
CORS	Nginx CORS headers restricting origin to known domains
Rate Limiting	Nginx limit_req_zone; 100 req/min per IP on /api/
Session Expiry	JWT 8-hour inactivity timeout; refresh token revoked on logout
Audit Trail	All expense status transitions in approval_logs; loan repayments in loan_repayments

7. Infrastructure & Deployment

7.1 Server Architecture

Component	Detail
Domain	thai.expenses.syaefulaz.my.id
Server	Texcoms VPS / cloud instance
OS	Ubuntu 22.04 LTS
Nginx	Reverse proxy on port 443 (SSL), port 80 (redirect to HTTPS)
NodeJS API	PM2 process manager, port 3000 (internal only)
Angular Build	Static files in /var/www/expenses-tracker/dist/
MySQL 8	Port 3306, accessible only via localhost
SSL	Let's Encrypt via Certbot, auto-renewal via cron

7.2 Nginx Configuration

```
server {

listen 80;

server_name thai.expenses.syaefulaz.my.id;
```

```
return 301 https://$host$request_uri;
}

server {
listen 443 ssl http2;
server_name thai.expenses.syaefulaz.my.id;

ssl_certificate /etc/letsencrypt/live/thai.expenses.syaefulaz.my.id/fullchain.pem;
ssl_certificate_key /etc/letsencrypt/live/thai.expenses.syaefulaz.my.id/privkey.pem;

location / {
root /var/www/expenses-tracker/dist;
try_files $uri $uri/ /index.html;
}

location /api/ {
proxy_pass http://127.0.0.1:3000;
proxy_http_version 1.1;
proxy_set_header X-Real-IP $remote_addr;
}
}
```

7.3 SSL Certificate Setup

Step	Command
Install Certbot	apt install certbot python3-certbot-nginx
Obtain Certificate	certbot --nginx -d thai.expenses.syaefulaz.my.id
Auto-renewal Check	certbot renew --dry-run
Nginx Reload on Renew	systemctl reload nginx

8. Project Structure

8.1 Angular Frontend Structure

```
expenses-tracker-ui/
|-- angular.json # Angular CLI config
|-- package.json # npm dependencies
```

```
+-- src/
|-- app/
| |-- core/ # Guards, interceptors, services
| |-- features/
| | |-- auth/ # Login, logout
| | |-- dashboard/ # Main dashboard (KPIs + loan widget)
| | |-- expenses/ # Expense CRUD pages
| | |-- loans/ # Loan dashboard (NEW)
| | |-- projects/ # Project management
| | |-- approvals/ # Approval queue
| | |-- reports/ # Export pages
| | +-- admin/ # User & settings (ADMIN only)
| +-- shared/ # Shared components, pipes, directives
|-- assets/
+-- environments/
```

8.2 NodeJS Backend Structure

```
expenses-tracker-api/
|-- package.json
|-- tsconfig.json
+-- src/
|-- main.ts # Application entry point
|-- app.module.ts # Root module (NestJS)
|-- auth/ # JWT auth, login, guards
|-- users/ # User management module
|-- expenses/ # Expense CRUD + workflow
|-- loans/ # Loan management (NEW)
|-- projects/ # Project management
|-- categories/ # Category config
|-- reports/ # PDF/Excel export
+-- common/ # Shared DTOs, filters, interceptors
+-- db/migration/ # Flyway migration scripts
```

8.3 Database Migration

Maven Phase	Command / Action
Apply migrations	mvn flyway:migrate
Baseline existing DB	mvn flyway:baseline
Show migration status	mvn flyway:info

9. Loan & Settlement Workflow (NEW)

This section describes the end-to-end loan and settlement workflow introduced in this version. It covers how loans are created, how repayment is declared, and how actual repayments are recorded.

9.1 Loan Creation Flow

Step	Actor	Action	System Response
1	Syaeful (borrower)	Fills expense form: amount=500 THB, source="Winda Cash"	Frontend validates source != current user
2	Syaeful	Submits expense (status → PENDING)	API creates expense record
3	System (auto)	Detects source="Winda Cash" → lender = Winda	API atomically creates loan: lender=Winda, borrower=Syaeful, amount=500 THB, status=UNSETTLED
4	Winda (lender)	Opens /loans dashboard	Winda sees "Syaeful owes me 500 THB" card
5	System	Records in audit log	approval_logs entry created

9.2 Repayment Declaration and Recording Flow

Step	Actor	Action	System Response
1	Syaeful	Informs Winda: "I will repay 1500 THB" (verbal/chat)	No system action — social agreement only
2	Winda (lender)	On /loans dashboard, clicks "Declare Repayment" on Syaeful's loan	Enters declared_repayment = 1500 THB
3	System	Updates loan: declared_repayment=1500, remaining_balance=1500	Dashboard updates — shows 1500 THB outstanding
4	Syaeful	Makes partial payment: pays 500 THB cash to Winda	Offline action — payment happens in person
5	Winda (lender)	On /loans dashboard, clicks "Record Repayment" on Syaeful's loan	Enters actual_repaid_increment = 500 THB

Step	Actor	Action	System Response
6	System	Updates loan: actual_repaid=500, remaining_balance=1000	loan_repayment record created; dashboard shows 1000 THB remaining
7	Repeat	Steps 4–6 repeat until fully settled	System marks status=FULLY_SETTLED when actual_repaid >= declared_repayment

9.3 Loan Status State Machine

Status	Trigger	Dashboard Display
UNSETTLED	Loan created; no repayments recorded	Amber badge: "Unsettled — 0 repaid / 0 declared"
PARTIAL	actual_repaid > 0 but < declared_repayment	Yellow badge: "Partial — X repaid / Y declared"
FULLY_SETTLED	actual_repaid >= declared_repayment	Green badge: "Settled"
OVERPAID	actual_repaid > declared_repayment	Blue badge: "Overpaid by X THB" (informational)

10. Design Acceptance Criteria

This design will be considered approved when all of the following criteria are met:

#	Acceptance Criterion	Verification Method
DAC-01	Angular app serves from Nginx at / with clean URLs (no # hash)	Browser test: navigate to /expenses directly
DAC-02	API accessible at https://thai.expenses.syaefulaz.my.id/api/v1/	curl -k https://.../api/v1/health returns 200
DAC-03	HTTPS SSL certificate is valid and auto-renews	SSL Labs test: grade A
DAC-04	JWT login flow: POST /auth/login returns token; protected routes reject unauthenticated requests	API integration test
DAC-05	All database tables created via MySQL migrations (including loans and loan_repayments)	mvn flyway:info shows all migrations applied
DAC-06	RBAC enforced: PM cannot access /admin/*; DIR read-only; ADMIN full access	Role-based API access test
DAC-07	Budget alert at 80% and 100% displayed in dashboard	Test: create expenses to reach threshold
DAC-08	PDF and Excel exports produce readable output	Manual test: download and open files
DAC-09 [NEW]	Expense with source belonging to another user auto-creates a loan visible on lender's dashboard	Create expense with source=Winda Cash as Syaeful → check /loans as Winda
DAC-10 [NEW]	Lender can declare expected repayment amount; remaining balance updates correctly	PUT /api/v1/loans/:id/declare → GET loan → verify remaining_balance
DAC-11 [NEW]	Lender can record partial repayment; loan_repayment audit record created	PUT /api/v1/loans/:id/repay → GET /api/v1/loans/:id → verify loan_repayments array
DAC-12 [NEW]	Dashboard loan widget shows mirrored outstanding balances for both lender and borrower	Create loan → verify both dashboards show consistent data

End of Document — DD-TXC-EXP-2025-001 v1.0
TEXCOMS WORLDWIDE — CONFIDENTIAL